Overview #3

1. **Summary of what was in this segment:**
    a. In this segment, I made/developed two deep learning models using PyTorch and evaluated their ability to predict Integrated Primary Production (IPP) values. The models were compared against the best traditional machine learning model from Overview #2, which was a tuned XGBoost model achieving an R2 score of 0.6455. The goal for this segment was to see if and how good deep learning models would perform when compared to the previous model we used, which was XGBoost.
2. **Deep Learning Models:**
    a. Model 1: Simple Deep Neural Network (DNN)
        i. This was a straightforward, simple DNN with 4 layers. The input layer accepts 6 features and passes them through progressively smaller hidden layers before producing a single output prediction (log_IPP). Throughout this NN, I used the RELU activation function at each layer to introduce non-linearity. This was allowing the model to capture complex relationships between features and IPP.

Layer	Input Neurons	Output Neurons	Activation
Input → Hidden 1	6	64	ReLU
Hidden 1 → Hidden 2	64	32	ReLU
Hidden 2 → Hidden 3	32	16	ReLU
Hidden 3 → Output	16	1	None (linear)

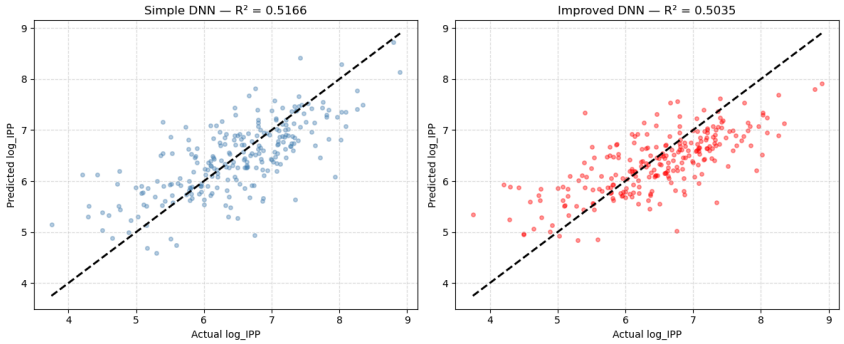
    b. Model 2: Improved DNN with Dropout and Batch Normalization
        i. The second DNN was built with larger hidden layers and two regularization techniques, Batch Normalization and Dropout, to prevent overfitting and improve generalization.
        ii. Batch normalization: Normalizes the output of each layer so all neurons operate on the same scale. This stabilizes training and helps the model learn faster.
        iii. Dropout: Randomly disables a percentage of neurons during each training step. This forces the remaining neurons to learn independently, preventing the model from memorizing the training data.

Layer	Input	Output	BatchNorm	Dropout	Activation
Input → Hidden 1	6	128	Yes	30%	ReLU
Hidden 1 → Hidden 2	128	64	Yes	20%	ReLU
Hidden 2 → Hidden 3	64	32	Yes	10%	ReLU
Hidden 3 → Output	32	1	No	No	None

3. **Training of the models:**
    a. Throughout the training of both models, I used many different combinations of settings to see which one suited the best, and the table below shows the best setting at which I got the best results for both models.

Parameter	Simple DNN	Improved DNN
Epochs	300	500
Learning Rate	0.001	0.001
Batch Size	32	32
Optimizer	Adam	Adam
Loss Function	MSE (Mean Squared Error)	MSE (Mean Squared Error)
LR Scheduler	ReduceLROnPlateau (patience=20)	ReduceLROnPlateau (patience=20)
Random Seed	42	42

4. **Results:**
    a. Here is the graph i created once I scored both models:



    b. The table below shows the R^2 and RMSE scores for both models.

Model	R2 Score	RMSE	Notes
Simple DNN (PyTorch)	**0.5166**	**0.6240**	**Best deep learning result**
Improved DNN (PyTorch)	0.5035	0.6323	Unstable training on small data

    c. Comparing All Models:
        i. The table below summarizes the performance of all models trained, ranked by R^2 scores.

Rank	Model	R2 Score	RMSE
1	**XGBoost Tuned**	**0.6455**	**0.5344**
2	Random Forest	0.5927	0.5727
3	Simple DNN (PyTorch)	0.5166	0.6240
4	Improved DNN (PyTorch)	0.5035	0.6323
5	MLP Neural Network (sklearn)	0.4708	0.6528
6	Linear Regression (Baseline)	0.3593	0.7184

5. **Why XGBoost performed better than Deep Learning:**
    a. Throughout this segment and the previous ones, a key insight is that the Deep Learning Models performed worse than XGBoost on this dataset. This outcome, while initially unexpected, is consistent with established findings in the machine learning literature regarding tabular data with limited samples. The three key reasons why this performance happened with the deep learning models could be:
        i. **Small dataset size (1,244 rows):** Deep learning models generally require tens of thousands of training samples to reach their full potential. With only 1,244 unique stations, the neural networks did not have enough data to learn the underlying patterns reliably. XGBoost, by contrast, is specifically designed to perform well on small to medium-sized tabular datasets.
        ii. **XGBoost is a tabular data specialist:** Gradient boosting methods like XGBoost have been consistently shown to outperform deep learning on structured tabular data. Deep learning tends to shine on unstructured data such as images, text, and audio.
        iii. **Regularization backfired on Improved DNN:** The Dropout and Batch Normalization added to the Improved DNN, while useful for larger datasets, introduced additional difficulty for the model to converge on only 1,244 rows. This explains why the Improved DNN (R2=0.50) actually performed worse than the simpler architecture (R2=0.52)
6. **Conclusion:**
    a. Overall, the deep learning methods worked, but they performed poorer when compared to the XGBoost model, which was used previously. So, in this particular scenario, I think that using XGBoost would be better for us as it performed better and achieved a high metric score.
    b. Key Findings:
        i. Deep learning models achieved reasonable but lower performance (Simple DNN R2=0.5166), consistent with expectations for a dataset of this size.
        ii. The Simple DNN outperformed the Improved DNN, suggesting that regularization techniques like Dropout and Batch Normalization are not beneficial when training data is limited.
        iii. The performance gap between XGBoost and deep learning would likely narrow significantly with a larger dataset.
        iv. All models consistently identified log_Chlavg (average chlorophyll within the euphotic zone) as the most important predictor of IPP.
    c. Next Steps:
        i. Incorporate additional datasets to increase training samples beyond 1,244 stations.
        ii. Explore more advanced deep learning architectures.
        iii. Investigate ensemble methods combining XGBoost and Random Forest predictions.